

Write-up

Architecture

```
operator prompt -> local LLM (Ollama) -> validated JSON mission plan
                -> formation layer -> deterministic executor -> PyBullet sim
```

The full stage diagram and data-contract table live in `docs/architecture.md`; the summary that matters for grading purposes is the boundary discipline.

1. `llm_planner/planner.py` is the only file that talks to an LLM (a local Ollama server over HTTP). It has one function, `plan_mission(prompt) -> str`, and is never called again once the mission JSON exists there is no code path from the model back into vehicle control.
2. `schema/validator.py` is a hard gate, not a warning. `validate_mission()` runs JSON Schema structure checks plus cross-field safety checks (altitude/speed ceilings, spacing vs. minimum separation, geofence containment, degenerate-polygon rejection) on the LLM's raw output before anything downstream sees it. A second stage, `validate_formation_separation / validate_formation_geofence`, re-checks the *concrete per-drone paths* after the formation layer computes them because “do any two drones get too close” isn't answerable from the raw mission JSON alone. Either stage failing stops the pipeline and prints why; nothing partially executes.
3. `formation/` is pure geometry (`{agent_id: [(x, y), ...]}` in, plain floats out) no PyBullet types, unit-testable without importing the simulator at all.
4. `executor/executor.py` is the only place mission JSON meets the simulator, and even there it only calls `sim.run_simulation()` with fully-formed trajectories; `sim/sim_runner.py` doesn't know where they came from.

Every arrow between these stages is a plain-data boundary (strings, dicts, tuples) nothing upstream of the executor imports `gym_pybullet_drones`, and nothing downstream of `llm_planner` imports or knows about an LLM. That separation is what makes the Ollama swap described below a one-file change instead of a rewrite.

Which Challenge Was Attempted

Multi-agent formations fully implemented and demonstrated

Two prompts were implemented, run end to end through the real pipeline (`--prompt` → Ollama → validation → formation → `--gui` PyBullet), and recorded as the two demo videos in `demo/`.

- **Wedge-formation sweep** “You three sweep this area in a wedge formation” → `formation.type: wedge, task.type: area_sweep`, 3 agents holding a wedge (lateral + longitudinal rank offsets, see `docs/design_decisions.md` §8) along a single-pass sweep line through

the target area.

- **Split-route-and-regroup** “Split this patrol route between the two of you and regroup at the end” → `formation.type: split, task.type: patrol_route`, 2 agents flying distinct, distance-split segments of the same route that reconverge at `task.regroup_point` (`formation/route_split.py`, §9 in design decisions).

line and column formations are also implemented and unit-tested (`tests/test_formation_geometry.py`) but weren’t separately recorded as demo videos, since they share the same rank-offset machinery as `wedge` see `formation/geometry.py`.

Scaling Story: From This Take-Home to a Harder, Real-World System

- **Sim → real flight controller.** `sim/sim_runner.py` talks to gym-pybullet-drones’ `CtrlAviary` + `DSLPIDControl`, a pure-Python rigid-body simulation with no radio link, no ESCs, no real IMU. Moving toward real hardware, the natural next step is [PX4 SITL](#) in place of PyBullet’s physics (still a simulation, but flying the *actual* flight-controller firmware over a real MAVLink link), then swapping in [MAVSDK](#) as the executor’s output target instead of `sim.run_simulation()` for real airframes. Because `executor/executor.py` already produces plain per-agent XYZ target trajectories rather than reaching into simulator internals (§10, determinism-by-construction), this is a swap of what consumes those trajectories, not a rewrite of what produces them.
- **Local Ollama → hosted/faster LLM.** The current local `qwen2.5:7b` path costs roughly two minutes of wall-clock time per schema-constrained generation on CPU-only hardware (see README “Reproducibility notes” for the measured figure) fine for a one-shot planning call in a demo, not fine for anything closer to real-time re-planning. `llm_planner/planner.py` is a single HTTP call behind one function (`plan_mission()`); pointing it at a hosted API (starting with reverting to the original Anthropic-API call this project was scoped against see below) or a GPU-served local model cuts that to low-single-digit seconds without touching any other file, by the same boundary argument as the Ollama swap itself.
- **More agents.** `mission_schema.json` currently caps `num_agents` at 2–3 (minimum/maximum) to match this challenge’s stated scope. The formation math itself (`formation/geometry._rank_offsets`) is already rank-indexed and doesn’t hard-code a drone count, so raising that cap is mostly a schema-limit change the real scaling cost shows up in `schema.validate_formation_separation`’s pairwise distance check, which is $O(n^2)$ in agent count per waypoint index and would want a spatial index (e.g. a k-d tree per timestep) well before agent counts reach the dozens.
- **Real sensor noise.** `sim/sim_runner.py` runs PyBullet’s `Physics.PYB` mode deterministically off ground-truth state no IMU noise, no GPS drift, no downwash/ground-effect perturbation. A harder version of this problem would run gym-pybullet-drones’ aerodynamic physics modes (or real SITL sensor models) and feed the formation/separation checks *estimated* state instead of ground truth, which changes `min_separation_m` from a hard pass/fail into something that needs a safety margin sized to the estimator’s actual error, not just the nominal spacing.
- **Comms latency / dropout between drones.** Nothing in this pipeline currently models inter-drone communication at all `executor.py` synchronizes every agent’s trajectory centrally,

before any drone moves (§11, slowest-agent time-sync), which is only valid because a single process has perfect, instant knowledge of every agent’s plan. A real multi-drone deployment needs that assumption replaced with an explicit comms model: bounded-latency or lossy links between drones (or between drones and a ground station), each agent re-planning locally from possibly-stale knowledge of its squadmates’ positions, and formation-hold logic that degrades gracefully (e.g. widen `min_separation_m`, or fall back to independent waypoint-following) rather than assuming instant consensus the way the current lockstep executor does.

Ollama Instead of the Anthropic API (Design Decision)

The task spec calls for the Anthropic API as the LLM layer, and that’s what `llm_planner/planner.py` originally called. It was switched to a locally-hosted model served by [Ollama](#) (default `qwen2.5:7b`) because no Anthropic API credit was available to run this system end to end. This is a documented substitution, not a silent scope change full reasoning, what changed (only `llm_planner/planner.py`; a plain `stdlib` HTTP call to `POST /api/chat` in place of the Anthropic SDK, plus a JSON-parse retry loop that 7-8B local models need and Claude’s structured-output enforcement didn’t), what didn’t change (`mission_schema.json`, `schema/`, `formation/`, `executor/`, `sim/` are all untouched and don’t know which backend produced the mission JSON), and exactly what reverting to the Anthropic API would take (a single-file change, restoring the original `client.messages.create(..., output_config.format=schema)` call) is in `docs/design_decisions.md` §14.

Cited Sources

Source	License	What was used
gym-pybullet-drones , pinned to commit 9bc12bc	MIT (Copy- right © 2020 Jacopo Panerati)	The simulation backend itself. <code>sim/sim_runner.py</code> adapts the <code>CtrlAviary</code> + <code>DSLPIDControl</code> + <code>Logger</code> control-loop pattern from the project’s own <code>examples/pid.py</code> specifically the class combination and import paths, the <code>env.step()</code> ordering (use the previous action, then compute the next action from the returned <code>obs</code>), and the <code>target_pos/target_rpy</code> keyword shape passed to <code>DSLPIDControl.computeControlFromState</code> . Everything else in <code>sim/</code> and <code>executor/</code> (trajectory construction from mission JSON, initial-position sourcing, loop tiling, the min-separation report) is original. Pinned to this commit rather than the <code>v1.0.0</code> tag because that tag predates the Gymnasium-based rewrite (5-tuple <code>env.step()</code> , current <code>CtrlAviary</code> / <code>DSLPIDControl</code> / <code>Logger</code> APIs) this project depends on see <code>docs/design_decisions.md</code> §13.
Ollama + qwen2.5:7b	Ollama itself is MIT; Qwen2.5 is Apache 2.0 (per its Ollama library page)	Local LLM serving, substituted for the Anthropic API see “Ollama Instead of the Anthropic API” above and §14.
jsonschema >=4.23)	(PyPI, MIT	<code>schema/validator.py</code> ’s structural validation of mission JSON against <code>mission_schema.json</code> the single source of truth for the mission contract (chosen over a parallel Pydantic model; see §12).
pytest (PyPI, >=8.0)	MIT	Test runner for <code>tests/</code> (schema validator + formation geometry/splitting logic).
Anthropic API (<code>output_config.format</code> / structured outputs)	N/A (API, not a repo)	The originally-specified LLM backend for this task; not currently called (no API credit available), but the schema-constrained-decoding <i>pattern</i> it motivated (§1, §2, §5 in design decisions why the schema uses <code>anyOf</code> instead of <code>if/then</code> , why fields are required-but-nullable, why structured outputs is treated as a convenience and not the safety mechanism) is preserved and reused as Ollama’s own <code>format</code> field.

Full rationale for every non-obvious choice above (not just the citations) is in `docs/design_decisions.md`.